

1. Общая концепция

Kouki — это модульная платформа для оркестрации команд, предназначенная для встраивания в AI-агенты, голосовые ассистенты, CLI-утилиты, веб-сервисы и автоматизированные системы. Она состоит из трёх основных уровней:

- **Mugura** — чистое ядро оркестрации. Регистрирует команды через атрибуты, управляет их жизненным циклом, вызывает хендлеры в изолированных DI-скоупах. Не зависит от окружения (файловая система, терминал, сеть).
- **Shimi** — слой абстракций и адаптеров. Определяет интерфейсы для работы с окружением (файловая система, терминал, хранилище, логирование) и предоставляет механизм контекстов выполнения (`IExecutionContext`, `IContextRegistry`). Разделён на `Abstractions` (только интерфейсы) и `Context` (реализации реестра и базового контекста).
- **Ubusuna** — библиотека расширений, содержащая готовые адаптеры (консольные, HTTP, FTP), конкретные контексты, дополнительные роутеры (`RemoteRouter`, `MushiRouter`, `DelegatingRouter`) и фабрики. Подключается одной строкой через `AddUbusuna()`. //todo: delegatingrouter по находится в mugura

Такое разделение обеспечивает гибкость, переиспользуемость и тестируемость.

2. Mugura (ядро)

2.1. Назначение

- Регистрация команд через атрибут `[Command]` на классах-хендлерах.
- Оркестрация вызова: от имени команды до выполнения хендлера.
- Управление временем жизни: каждый вызов выполняется в своём DI-скоупе (изоляция зависимостей).
- Единый формат ввода/вывода: входной аргумент — `JsonNode?`, выход — `JsonNode` (может быть любым JSON-совместимым значением). Внутри ядра аргумент десериализуется в строго типизированный DTO команды.
- Базовая валидация DTO через `System.ComponentModel.DataAnnotations` (с возможностью расширения кастомными атрибутами из внешних библиотек).

2.2. Ключевые компоненты

- **CommandAttribute** — применяется к хендлеру. Содержит имя команды, описание, категорию (битовые флаги), флаг подтверждения и пример.
- **CommandRegistry** — сканирует сборки, находит хендлеры с атрибутом, извлекает DTO команды, собирает правила валидации из атрибутов свойств DTO. Хранит записи `CommandEntry` в словарях по имени и по типу DTO.
- **CommandEntry** — запись с полями: `CommandType`, `HandlerType`, `Name`, `Description`, `Category`, `ValidationRules`, `Example`.

- **ICommandFactory** — фабрика для создания DTO из **JsonNode** . Содержит generic-метод **CreateCommand<TCommand>** ; сама фабрика не generic. Использует **JsonSerializer.Deserialize** для десериализации.
- **ICommandValidator<TCommand>** — интерфейс валидатора. Реализация по умолчанию — **DataAnnotationsValidator<TCommand>** , использующая правила из **CommandEntry** (полученные через **CommandRegistry**). Может быть переопределена для конкретной команды через DI.
- **IScopeManager / ScopeManager** — управляет созданием DI-скоупов для каждого вызова команды.
- **ICommandDispatcher** — generic-метод **SendAsync<TCommand>** , внутри скоупа получает хендлер **ICommandHandler<TCommand>** и вызывает его.
- **IRouter** — интерфейс для маршрутизации. Основная реализация — **StaticRouter** .
- **StaticRouter** — локальный исполнительный роутер. По имени команды находит **CommandEntry** , создаёт DTO через фабрику, применяет валидатор (если зарегистрирован), затем вызывает диспетчер. Кеширует делегаты для вызова валидатора и диспетчера по типу команды для производительности.
- **DelegatingRouter** — абстрактный базовый класс для роутеров-обёрток, которые делегируют выполнение внутреннему роутеру. Упрощает создание новых типов роутеров (например, **IntentRouter** , **RemoteRouter**).
- **Метод расширения AddMugura** — регистрирует все компоненты ядра в DI (синглтоны), сканирует сборки и регистрирует хендлеры как **Scoped** .

2.3. Поток вызова (статический роутер)

1. Внешний код вызывает **IRouter.RouteAsync(commandName, argument, context?)** .
2. **StaticRouter** получает **CommandEntry** из **CommandRegistry** .
3. Фабрика создаёт DTO команды из **JsonNode** .
4. Если зарегистрирован валидатор, он проверяет DTO; при ошибке возвращается **JsonNode** с ошибкой.
5. Диспетчер создаёт скоуп, получает хендлер и вызывает **HandleAsync** .
6. Хендлер выполняет бизнес-логику, используя контекст (если передан) и сервисы из DI.
7. Результат (**JsonNode**) возвращается обратно.

3. Shimi (абстракции и контексты)

3.1. Абстракции (Shimi.Abstractions)

Проект содержит только интерфейсы, без реализаций. Основные из них:

- **IFileSystem** — базовый интерфейс для работы с байтовыми данными: **ReadBytesAsync** , **WriteBytesAsync** , **ExistsAsync** , **GetTreeAsync** .
- **ITextFileSystem** — расширяет **IFileSystem** , добавляет методы для текстовых файлов (**ReadTextAsync** , **WriteTextAsync** , **AppendTextAsync**) с параметром **Encoding** .

- **IJsonFileSystem** — расширяет **IFileSystem**, добавляет методы для JSON-сериализации (**ReadJsonAsync**, **WriteJsonAsync**) с параметрами **JsonSerializerOptions**.
- **IStateStore** — хранилище ключ-значение для состояния.
- **ITelemetryLogger** — логирование событий и ошибок.
- **IExecutionContext** — контекст выполнения, содержащий конкретные реализации **IFileSystem**, **ITerminal**, **IStateStore** и доступ к дополнительным сервисам через **GetService<T>()**. Имеет свойство **Name** для идентификации.
- **IContextRegistry** — реестр предопределённых контекстов. Позволяет регистрировать контексты по имени, устанавливать и получать контекст по умолчанию.

3.2. Контексты (Shimi.Context)

//todo Содержит реализации:

- **ContextRegistry** — синглтон, хранит словарь контекстов. Методы: **RegisterContext**, **GetContext**, **SetDefaultContext** (по экземпляру или по имени), **GetDefaultContext**.
- **BaseExecutionContext** — абстрактный базовый класс для контекстов. Хранит имя и словарь дополнительных сервисов. Предоставляет реализацию **GetService<T>()**. Конкретные контексты должны переопределять свойства **FileSystem**, **Terminal**, **StateStore**.
- **InMemoryStateStore** — базовая реализация **IStateStore** в памяти (опционально).
- **Метод расширения AddShimiContext** — регистрирует **IContextRegistry** и **IStateStore** (можно переопределить).

Важно: **Shimi.Context** **не создаёт** дефолтный контекст автоматически. Приложение должно явно установить его через **SetDefaultContext**.

4. Ubusuna (библиотека расширений)

Kouki.Ubusuna предоставляет готовые реализации адаптеров, контекстов, роутеров и фабрик, чтобы упростить подключение и настройку.

4.1. Адаптеры

//todo

- **ConsoleFileSystem** — реализует **IFileSystem** (и опционально **ITextFileSystem**, **IJsonFileSystem**) для работы с локальной файловой системой через стандартные методы **System.IO**. Поддерживает работу с корневым каталогом.
- **ConsoleTerminal** — реализует **ITerminal**, запускает внешние процессы через **System.Diagnostics.Process**.
- **HttpFileSystem**, **FtpFileSystem** — (планируются) для работы с удалёнными файловыми хранилищами.

4.2. Контексты

- **ConsoleExecutionContext** — конкретный контекст с консольными адаптерами (`ConsoleFileSystem`, `ConsoleTerminal`, `InMemoryStateStore`).
- **ObsidianExecutionContext** — (планируется) для интеграции с Obsidian API.

4.3. Роутеры

//todo

- **DelegatingRouter** — абстрактный базовый класс (может быть перенесён в ядро или оставлен в Ubusuna). Реализует **IRouter**, принимает внутренний роутер в конструкторе и делегирует ему выполнение. Наследники могут переопределить логику.
- **RemoteRouter** — отправляет команды по сети (HTTP, gRPC) на удалённый сервер.
- **MushiRouter** — композитный роутер, последовательно пробует список внутренних роутеров до первого успешного выполнения.

4.4. Фабрики и DI

- **AddUbusuna(this IServiceCollection, Action<UbusunaOptions>?)** — регистрирует все стандартные адаптеры, контексты и роутеры. Позволяет через опции выбрать, какие адаптеры использовать, зарегистрировать предопределённые контексты и настроить роутинг.

Пример использования:

```
services.AddShimiContext();
services.AddUbusuna(options =>
{
    options.UseConsoleAdapters();
    options.SetDefaultContext("console");
    options.AddContext("admin", adminFs, adminTerminal);
    options.UseMushiRouter(); // или UseRemoteRouter, UseStaticRouter
});
```

5. Взаимодействие с голосовым ассистентом (IntentRouter)

//TODO **IntentRouter** — это внешняя реализация **IRouter**, живущая в приложении (например, в `Kagedama.Voice`). Она использует каскадную систему определения интента:

- **ML.NET / fastText** — быстрый уровень, ранжирует все интенты, выдаёт топ-10 кандидатов.
- **BERT / Sentence-BERT** — точный уровень, переранжирует кандидатов и выбирает лучший с оценкой уверенности.
- **LLM (GPT) или уточнение у пользователя** — резервный уровень, если уверенность BERT ниже порога.

`IntentRouter` извлекает слоты (аргументы) через Cross-Encoder или NER-модель, формирует `JsonNode` с аргументами и вызывает внутренний роутер (обычно `StaticRouter` или `MushiRouter`) для выполнения команды. Логирование отказов и успехов происходит через `ITelemetryLogger`.

6. Расширение роутинга (MushiRouter)

`MushiRouter` — композитный роутер, реализующий `IRouter`. Он принимает список внутренних роутеров и последовательно вызывает их до первого успешного выполнения. Это позволяет комбинировать локальный `StaticRouter`, `IntentRouter`, `RemoteRouter` и другие. Например, можно попробовать сначала `IntentRouter` (для голосовых команд), затем `RemoteRouter` (для удалённых), и только потом `StaticRouter`.

7. Преимущества разделения

- **Чистое ядро** — `Mugura` не зависит от окружения, легко тестируется, может использоваться в любых средах.
 - **Гибкие адаптеры** — `Shimi` предоставляет интерфейсы, а конкретные реализации подключаются через `Ubusuna` или приложение.
 - **Контексты выполнения** — позволяют переключать реализации сервисов (например, файловую систему) в зависимости от пользователя, проекта или окружения без изменения кода хендлеров.
 - **Модульные расширения** — `Ubusuna` предоставляет готовые компоненты, но их можно заменить своими.
 - **Единый формат ввода/вывода** (`JsonNode`) упрощает интеграцию.
 - **DI-дружественность** — всё собирается через стандартный контейнер.
-

8. Рекомендации по использованию

Для нового приложения:

1. Добавить ссылки на `Kouki.Mugura`, `Shimi.Abstractions`, `Shimi.Context`, `Kouki.Ubusuna`.
2. В DI вызвать `AddShimiContext()` и `AddUbusuna()` с нужными опциями.
3. Написать хендлеры, помеченные `[Command]`.
4. В точке входа (MCP, CLI, HTTP, голос) вызвать `IRouter.RouteAsync` с именем команды, `JsonNode` аргументами и, при необходимости, контекстом.

Для удалённого взаимодействия использовать `RemoteRouter` и `MushiRouter` для композиции.

9. Что не входит в архитектурную заметку

- Конкретные сигнатуры методов (они описаны в датафлоу и документации API).

- Детали реализации `CommandRegistry` (рефлексия или генерация кода).
- Подробности работы `ScopeManager`.
- Реализация ML-моделей для `IntentRouter`.

1. Базовые типы и форматы ICommand — маркерный интерфейс для DTO команд. Сам по себе не содержит свойств, служит для идентификации.

JsonNode — универсальный тип из System.Text.Json.Nodes, представляющий структурированные данные (объект, массив, строка, число, null). Используется для:

Входных аргументов команд (передаётся в IRouter.RouteAsync).

Выходных результатов хендлеров.

DTO команд — это record-классы, реализующие ICommand. Они содержат типизированные свойства (например, string Path, bool Recursive). Входной JsonNode десериализуется фабрикой в конкретный DTO.

IExecutionContext — контейнер для сервисов окружения (IFileSystem, ITerminal, IStateStore и опциональных). Имеет имя (Name) и может предоставлять дополнительные сервисы через GetService().

IContextRegistry — реестр, хранящий предопределённые контексты по строковому ключу. Позволяет устанавливать и получать контекст по умолчанию.

2. Регистрация команд CommandAttribute — применяется к классу хендлера. Содержит:

Name (обязательно)

Description (опционально)

Category (битовые флаги, например, FileSystem, Terminal, VoiceUI, Internal)

RequiresConfirmation

Example

CommandRegistry — сканирует сборки, находит все классы с атрибутом, определяет реализуемый ими интерфейс ICommandHandler, извлекает TCommand (тип DTO), собирает правила валидации из атрибутов свойств DTO (ValidationAttribute). Хранит записи CommandEntry в словарях по имени и по типу DTO.

CommandEntry — record с полями:

CommandType, HandlerType, Name, Description, RequiresConfirmation, Category (флаги), ValidationRules, Example.

3. Создание команды (фабрика) ICommandFactory — интерфейс с методом ICommand? Create(CommandEntry entry, JsonNode? argument, out string? error).

Реализация `CommandFactory`:

Содержит generic-метод `CreateCommand`, который десериализует `JsonNode` в `TCommand` через `JsonSerializer.Deserialize`.

Метод `Create` вызывает этот generic-метод через рефлекссию (или кешированный делегат) для типа `entry.CommandType`.

Если аргумент `null`, передаётся пустой объект `new JsonObject()`.

Возвращает созданный DTO или `null` с сообщением об ошибке.

Фабрика не использует внутреннего кеша — кеширование вызовов возложено на роутер.

4. Валидация команд `ICommandValidator` — интерфейс с методом `bool TryValidate(TCommand command, out string? error)`.

`DataAnnotationsValidator` — реализация по умолчанию:

В конструкторе получает `CommandRegistry` и извлекает `ValidationRules` для `TCommand`.

В `TryValidate` проверяет каждое свойство DTO на соответствие атрибутам `ValidationAttribute`.

Возвращает `false` и сообщение об ошибке, если хотя бы одно правило нарушено.

Валидатор регистрируется в DI как открытый generic (`ICommandValidator<>` → `DataAnnotationsValidator<>`).

При необходимости для конкретной команды можно зарегистрировать собственную реализацию `ICommandValidator`.

5. Управление скоупами `IScopeManager` — метод `Task RunInScopeAsync(Func<IServiceProvider, Task> action)`.

`ScopeManager` — реализация, зависящая от `IServiceScopeFactory`. Создаёт `AsyncServiceScope`, выполняет `action` и уничтожает `scope`.

6. Диспетчер команд `ICommandDispatcher` — метод `Task SendAsync(TCommand command, IExecutionContext? context = null, CancellationToken cancellationToken = default)`, где `TCommand : ICommand`.

`CommandDispatcher`:

Зависит от `IScopeManager`.

Внутри вызывает `_scopeManager.RunInScopeAsync`, где получает `ICommandHandler` из `IServiceProvider` и вызывает `HandleAsync`, передавая ему контекст (если он был передан).

7. Хендлер команд `ICommandHandler` — метод `Task HandleAsync(TCommand command, IExecutionContext? context = null, CancellationToken cancellationToken = default)`.

Конкретные хендлеры:

Помечены атрибутом `[Command]`.

Реализуют `ICommandHandler`.

В конструкторе получают глобальные зависимости (например, `ILogger`), но сервисы окружения получают через контекст (передаваемый параметром).

Используют `context.FileSystem`, `context.Terminal`, `context.StateStore` и т.д. для выполнения бизнес-логики.

8. Роутинг (статический и делегирующий) `IRouter` — метод `Task RouteAsync(string commandName, JsonNode? argument, IExecutionContext? context = null, CancellationToken cancellationToken = default)`.

`StaticRouter`:

Зависит от `CommandRegistry`, `ICommandFactory`, `ICommandDispatcher`, `IContextRegistry`.

Алгоритм:

Получает `CommandEntry` по имени. Если контекст не передан, получает дефолтный контекст из `IContextRegistry`. Создаёт DTO через фабрику. Получает валидатор (через DI и кеширование делегатов) и вызывает `TryValidate`. Вызывает диспетчер `SendAsync`, передавая контекст и команду. Кеширует делегаты для вызова валидатора и диспетчера по типу команды.

`DelegatingRouter` — абстрактный базовый класс для роутеров-обёрток:

Принимает `IRouter innerRouter` и `IContextRegistry` в конструкторе.

Реализует `RouteAsync`, делегируя вызов внутреннему роутеру (с подстановкой дефолтного контекста, если не передан).

Наследники могут переопределить `RouteAsync` для добавления своей логики (например, `IntentRouter` классифицирует фразу, а затем вызывает `base.RouteAsync`).

`MushiRouter` — композитный роутер, принимает список `IRouter` и последовательно вызывает их до первого успеха.

RemoteRouter — отправляет команды по сети, если не удаётся — делегирует внутреннему роутеру.

9. Контексты и реестр IContextRegistry — синглтон, хранит словарь контекстов. Методы:

IExecutionContext GetContext(string name) — возвращает контекст по имени, если не найден — дефолтный.

void RegisterContext(string name, IExecutionContext context).

void SetDefaultContext(IExecutionContext context) и void SetDefaultContext(string contextName) — устанавливают дефолтный контекст (по экземпляру или по имени).

IExecutionContext GetDefaultContext() — возвращает дефолтный контекст.

Реализация ContextRegistry выбрасывает исключение, если дефолтный контекст не установлен.

BaseExecutionContext — абстрактный базовый класс, хранит имя и словарь дополнительных сервисов. Конкретные контексты переопределяют свойства FileSystem, Terminal, StateStore.

10. Взаимодействие с Shimi (адаптеры) Интерфейсы окружения определены в Shimi.Abstractions:

IFileSystem — базовый (байтовые операции): ReadBytesAsync, WriteBytesAsync, ExistsAsync, GetTreeAsync.

ITextFileSystem — расширяет IFileSystem: ReadTextAsync, WriteTextAsync, AppendTextAsync с параметром Encoding.

IJsonFileSystem — расширяет IFileSystem: ReadJsonAsync, WriteJsonAsync с параметрами JsonSerializerOptions.

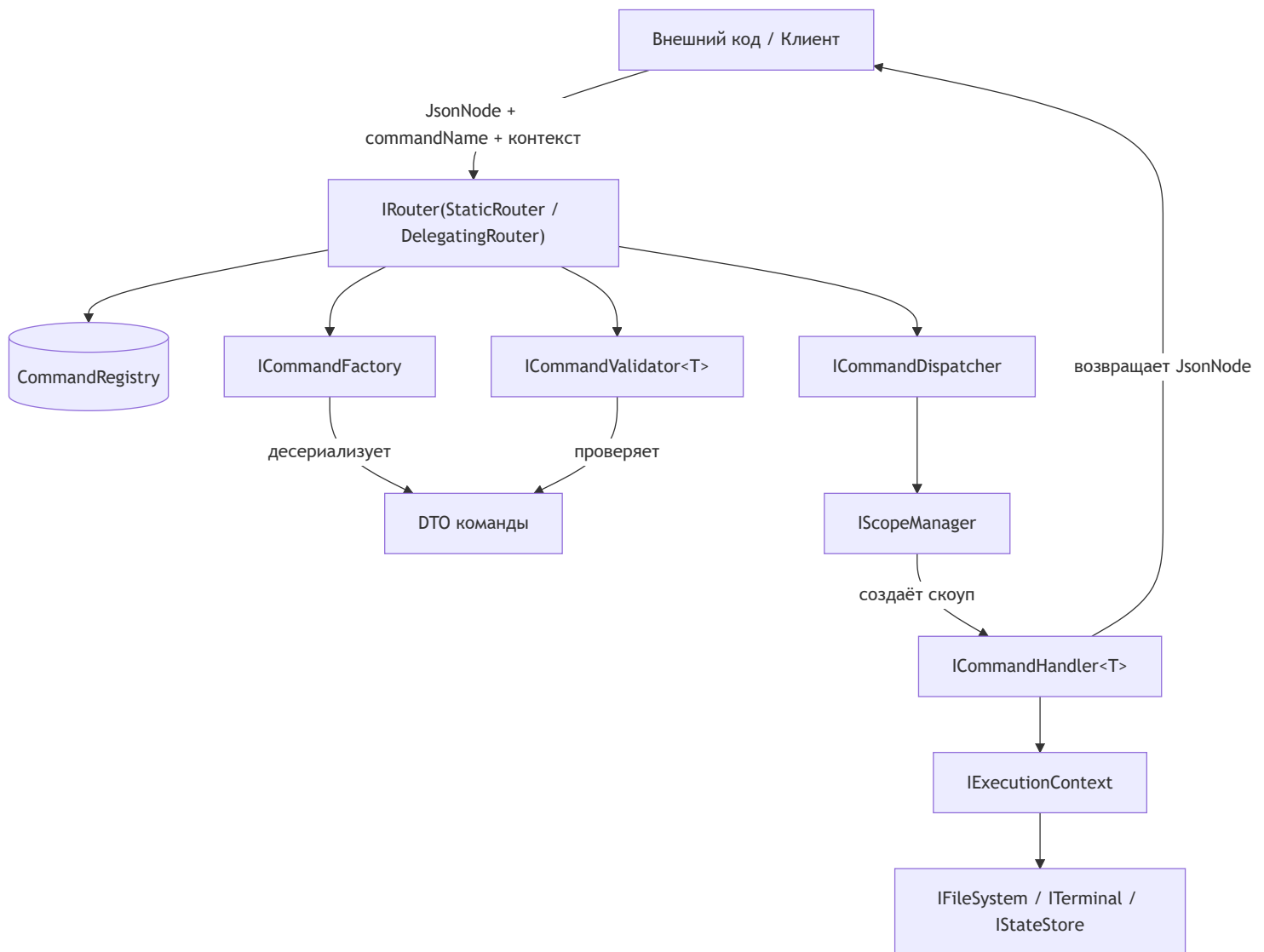
ITerminal — RunCommandAsync.

IStateStore — GetAsync, SetAsync, RemoveAsync.

ITelemetryLogger — LogEvent, LogError.

Конкретные адаптеры (ConsoleFileSystem, ConsoleTerminal) поставляются в Kouki.Ubusuna и регистрируются через метод расширения AddUbusuna.

11. Поток выполнения (полный цикл)



1. **Вход** — внешний код вызывает `IRouter.RouteAsync(commandName, argument, context?)`.
2. **Роутер** (например, `StaticRouter`) получает `CommandEntry` из `CommandRegistry`.
3. **Фабрика** создаёт DTO команды из `JsonNode` (через generic-метод `CreateCommand<TCommand>`).
4. **Валидатор** (если зарегистрирован) проверяет DTO; при ошибке возвращается `JsonNode` с ошибкой.
5. **Диспетчер** создаёт скоуп через `ScopeManager`, получает хендлер из DI и вызывает `HandleAsync`.
6. **Хендлер** выполняет бизнес-логику, используя сервисы из контекста (`IFileSystem`, `ITerminal`, `IStateStore`).
7. **Результат** (`JsonNode`) возвращается обратно вызывающему коду.

12. Расширение роутинга (IntentRouter) IntentRouter (внешняя реализация) переопределяет `RouteAsync`:

Принимает `commandName` как голосовую фразу.

Классифицирует интент (через каскад ML.NET → BERT → LLM).

Извлекает слоты (аргументы).

Формирует `JsonNode` с аргументами и вызывает `base.RouteAsync` с именем команды и аргументами.

При необходимости выбирает контекст на основе распознанного слота.

13. Логирование и телеметрия `ITelemetryLogger` используется в роутерах, диспетчере и хендлерах для логирования событий, ошибок и метрик. Например:

Успешные/неудачные выполнения команд.

Отказы интенгов (для дообучения моделей).

Долгие вызовы.

14. Примечания Все асинхронные методы поддерживают `CancellationToken`.

Входной аргумент всегда передаётся как `JsonNode` (может быть `null`, но нормализуется в пустой объект фабрикой).

Выход команды — `JsonNode`, что позволяет возвращать простые типы (`string`, `int`) или сложные объекты.

Жизненный цикл: `Registry`, `Factory`, `Dispatcher`, `Router`, `ScopeManager`, `ContextRegistry` — синглтоны; хендлеры и валидаторы — `Scoped` (создаются на каждый вызов команды).

Контекст (`IExecutionContext`) не является DI-сервисом и не привязан к скоупу; он передаётся явно как параметр.

Для повышения производительности `StaticRouter` кеширует делегаты вызова валидатора и диспетчера по типу команды.

Генерация кода (`Source Generators`) планируется для автоматической регистрации команд и валидаторов, что позволит отказаться от рефлексии в рантайме.

//TODO

//todo

Namespace Kouki.Mugura.Attributes

Classes

[CommandAttribute](#)

Enums

[CommandCategory](#)